Vidyavardhini's College of Engineering and Technology

Department of Artificial Intelligence & Data Science

AY: 2025-26

Class:	**TE**	**Semester:**	**VI**
Course Code:	**CSL601**	**Course Name:**	**Data Analytics and Visualization Lab**

Name of Student:	**Shravani Sandeep Raut**
Roll No.:	**40**
Experiment No.:	**07**
Title of the Experiment:	**Data Visualization: Use different R libraries for data visualization.**
Date of Performance:	**17/03/2026**
Date of Submission:	**24/03/2026**

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	1
Understanding	4-5	2-3	1
Journal work and timely submission	8-10	5-8	1-4

Checked by

Name of Faculty: Ms. Odilia Gonsalves

Signature:

Date:

Experiment No. 7

Aim - Data Visualization: Use different R libraries for data visualization.

Objective: - To understand and apply the R libraries for visualization.

Description: -

1. ggplot2

ggplot2 is an R data visualization library that is based on The Grammar of Graphics. ggplot2 can create data visualizations such as bar charts, pie charts, histograms, scatterplots, error charts, etc. using high-level API. It also allows you to add different types of data visualization components or layers in a single visualization. Once ggplot2 has been told which variables to map to which aesthetics in the plot, it does the rest of the work so that the user can focus on interpreting the visualizations and take less time in creating them. But this also means that it is not possible to create highly customized graphics in ggplot2. But there are a lot of resources in the RStudio community and Stack Overflow which can provide help in ggplot2 when needed. Just like dplyr, if you want to install ggplot2, you can install the tidyverse or you can just install ggplot2 using install.packages("ggplot2").

2. Plotly

Plotly is a free open-source graphing library that can be used to form data visualizations. Plotly is an R package that is built on top of the Plotly JavaScript library (plotly.js) and can be used to create web-based data visualizations that can be displayed in Jupyter notebooks or web applications using Dash or saved as individual HTML files. Plotly provides more than 40 unique chart types like scatter plots, histograms, line charts, bar charts, pie charts, error bars, box plots, multiple axes, sparklines, dendrograms, 3-D charts, etc. Plotly also provides contour plots, which are not that common in other data visualization libraries. In addition to all this, Plotly can be used offline with no internet connection. You can install Plotly from CRAN using install.packages('plotly') or install the latest development version from GitHub using devtools::install_github("ropensci/plotly").

3. Esquisse

Esquisse is a data visualization tool in R that allows you to create detailed data visualizations using the ggplot2 package. You can create all sorts of scatter plots, histograms, line charts, bar charts, pie charts, error bars, box plots, multiple axes, sparklines, dendrograms, 3-D charts, etc. using Esquisse and also export these graphs or access the code for creating these graphs. Esquisse is such a famous and easily used data visualization tool because of its drag and drops ability that makes it popular even among beginners. You can install Esquisse from CRAN using install.packages("esquisse") or install the development version from GitHub using remotes::install_github("dreamRs/esquisse").

4. Lattice

Lattice is a data visualization tool that is primarily used to implement Trellis graphs in R. These Trellis graphs are used to view many complicated and multi-variable data sets at the same time so they can be compared. Since all these different plots end up looking like a Trellis, this is called a Trellis graph. Since Lattice is a high-level data visualization library, it can handle many of the typical graphics without needing many customizations. In case you want to extend the capabilities of Lattice, they can download the LatticeExtra package which is an extended version. You can install Lattice from CRAN using install.packages("lattice") or install the development version from GitHub using remotes::install_github("deepayan/lattice").

5. RGL

The RGL package in R is created specifically for making 3-D data visualizations and data plots. It has many graphics commands that work in 3 dimensions but is modeled loosely after the classic 2-D graphics in R. RGL is also inspired by the grid package in R but it is incompatible with it. However, seasoned R coders can easily use RGL because of an existing familiarity with the grid. And RGL is very cool! It has a lot of options for 3-D shapes, various lighting effects, creating new shapes, and also animations. You can install RGL from CRAN using install.packages("rgl").

6. Dygraphs

The dygraphs package is an R interface to the JavaScript charting library dygraphs that are used to provide various charts for visualizing data sets. This package can be used for creating various interactive visualizations with zooming, and panning options along with default mouse-over labels. dygraphs also provides support for various graph overlays such as point annotations, shaded regions, event lines, etc. You can also plot the xts time series objects automatically. However, all of these features do not come at the expense of speed in dygraph. Rather, it can provide maximal interactivity even with millions of data points in the visualization. You can install RGL from CRAN using install.packages("dygraphs").

7. Leaflet

Just like dygraphs, the Leaflet package is an R interface to the JavaScript Leaflet library that is extremely popular. Leaflet is very useful in creating interactive but lightweight maps that are seen on various websites such as the Washington Post, the New York Times, etc. There are many useful features in this package such as interactive panning and zooming in the charts, the option to combine Polygons, Lines, Popups, etc. to create charts, embed maps in knitr, create maps in mercator projections that are non-spherical and so on. The Leaflet package can be used at the R console after installing it from CRAN using the command install.packages("leaflet").

All of these R Libraries for Data Visualization are excellent options if you want to create data visualizations. Each of these libraries has its strengths and you can choose the best one depending on the type of visualization or data science project you want to create. Now that you know these libraries, go on and create beautiful yet informative data visualizations using them!

Features of plotly and ggplot2 libraries are:

Features of Plotly:

- **Interactive Graphs** – Allows users to zoom, pan, hover for tooltips, and interact with data points.
- **Diverse Chart Options** – Supports line graphs, bar charts, scatter plots, heatmaps, 3D visualizations, and more.
- **Live Data Streaming** – Capable of visualizing real-time data with dynamic updates.

- **Extensive Customization** – Enables modifications in layout, colors, axes, and annotations.
- **Multiple Export Formats** – Allows saving visualizations in PNG, PDF, SVG, and HTML formats.

Features of ggplot2:

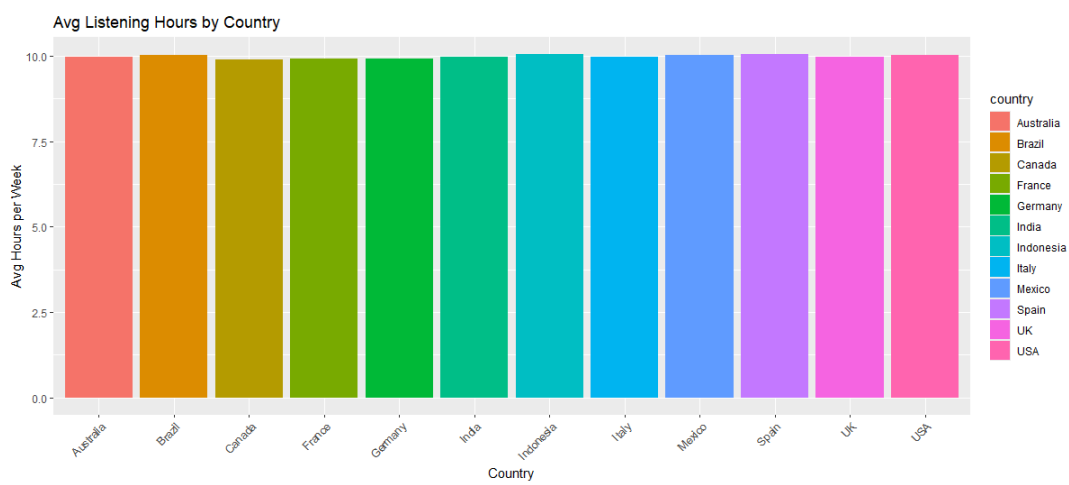
- **Structured Visualization Approach** – Implements the Grammar of Graphics for building plots layer by layer.
- **Extensive Customization** – Offers flexible theming, color schemes, and design modifications.
- **Variety of Plot Types** – Supports scatter plots, histograms, boxplots, violin plots, and more.
- **Faceting for Comparison** – Enables multi-panel plots using facet_wrap() and facet_grid().
- **Integrated Statistical Functions** – Provides built-in methods for trend lines, smoothing, and confidence intervals.

Dataset - Spotify User Behavior And Pattern

Program -

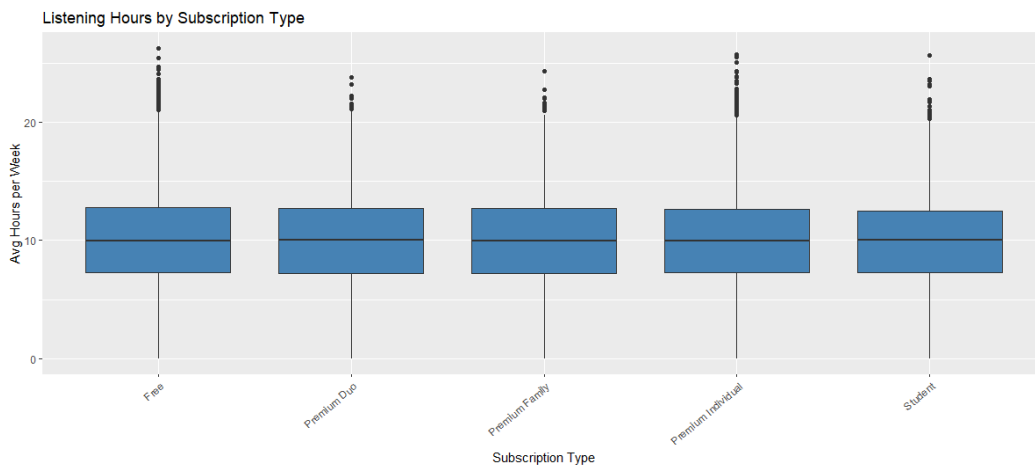
Write a R program to demonstrate use of the ggplot library.

```
library(ggplot2)
library(dplyr)
data <- read.csv("C:/Users/student/Desktop/TE/DAV/spotify_dataset.csv")

# 1. Bar plot: Average listening hours by country
ggplot(data, aes(x = country, y = avg_listening_hours_per_week, fill =
   country)) +
  geom_bar(stat = "summary", fun = "mean") +
  labs(title = "Avg Listening Hours by Country",
       x = "Country",
       y = "Avg Hours per Week") +
  theme(axis.text.x = element_text(angle=45, hjust=1))
```



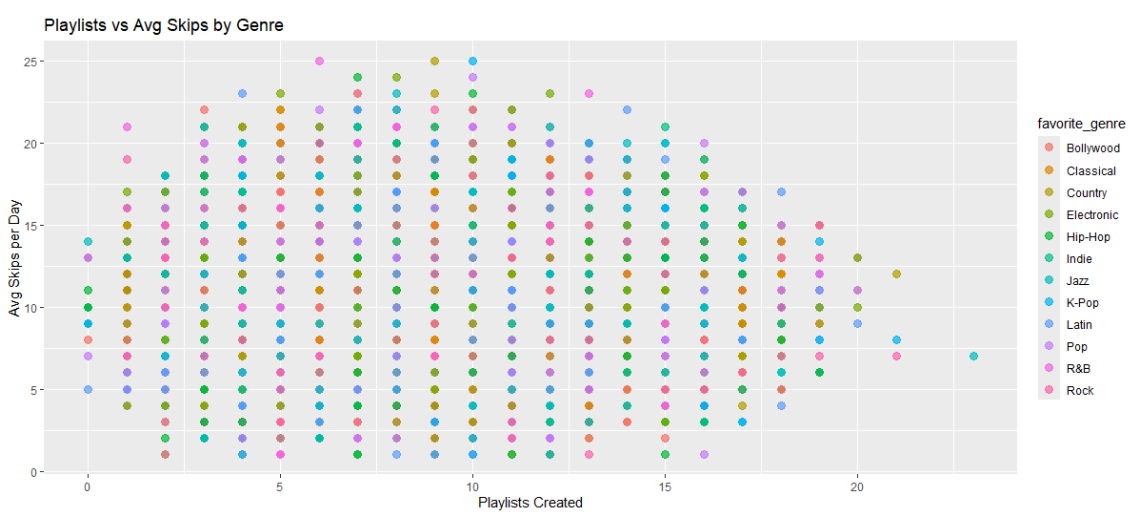
*Creates a bar chart where each bar represents the **average** listening hours per week of users from each country.*

```
# 2. Boxplot: Listening hours for different subscription types
ggplot(data, aes(x = subscription_type, y = avg_listening_hours_per_week))
  +
 geom_boxplot(fill = "steelblue") +
 labs(title = "Listening Hours by Subscription Type",
      x = "Subscription Type",
      y = "Avg Hours per Week") +
 theme(axis.text.x = element_text(angle = 40, hjust = 1))
```



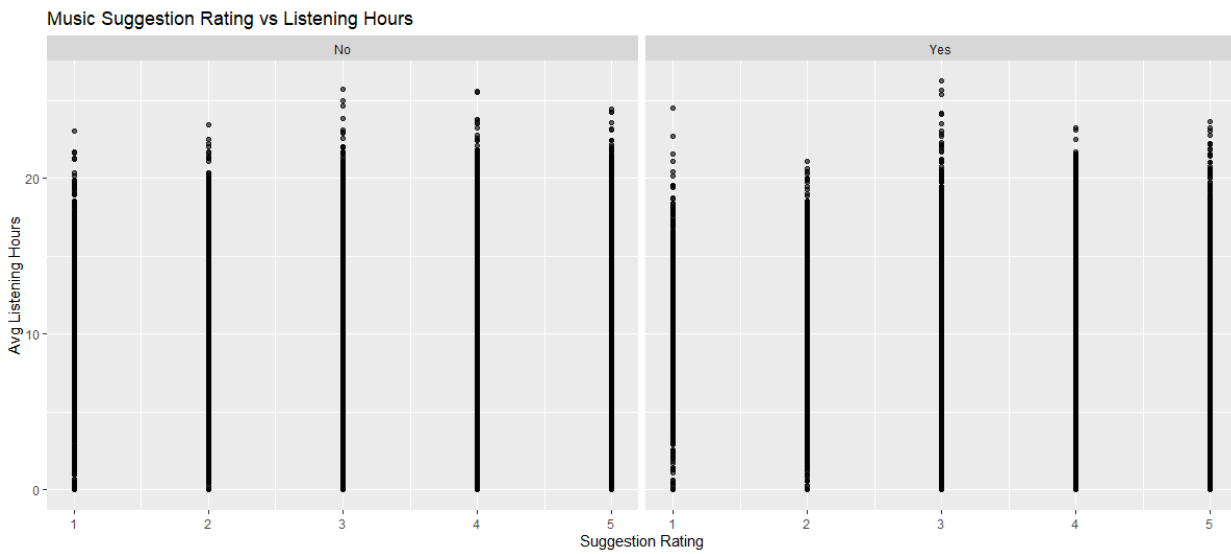
Shows distribution of average listening hours grouped by the subscription type using boxplots.

```
# 3. Scatter: Playlists Created vs Avg Skips per Day
ggplot(data, aes(x = playlists_created, y = avg_skips_per_day, color =
   favorite_genre)) +
 geom_point(alpha = 0.7, size = 3) +
 labs(title = "Playlists vs Avg Skips by Genre",
      x = "Playlists Created",
      y = "Avg Skips per Day")
```



A scatter plot where each point represents a user.

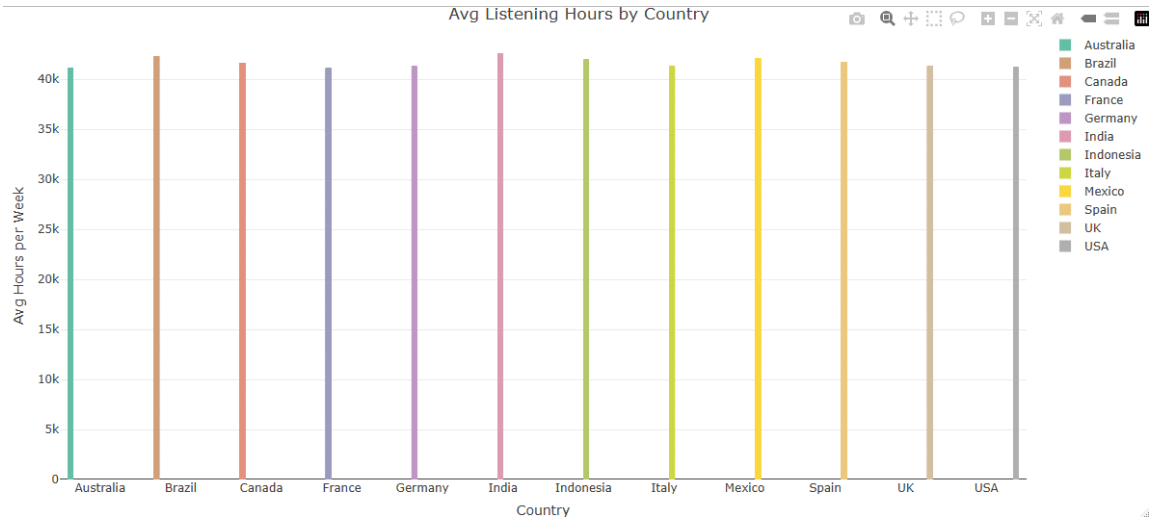
```
# 4. Faceted plot: Rating vs Avg Listening Hours split by Ad Interaction
ggplot(data, aes(x = music_suggestion_rating_1_to_5, y =
   avg_listening_hours_per_week)) +
  geom_point(alpha = 0.6) +
  facet_wrap(~ ad_interaction) +
  labs(title = "Music Suggestion Rating vs Listening Hours",
       x = "Suggestion Rating",
       y = "Avg Listening Hours")
```



Scatter plot of music suggestion ratings (1 to 5) vs average listening hours.

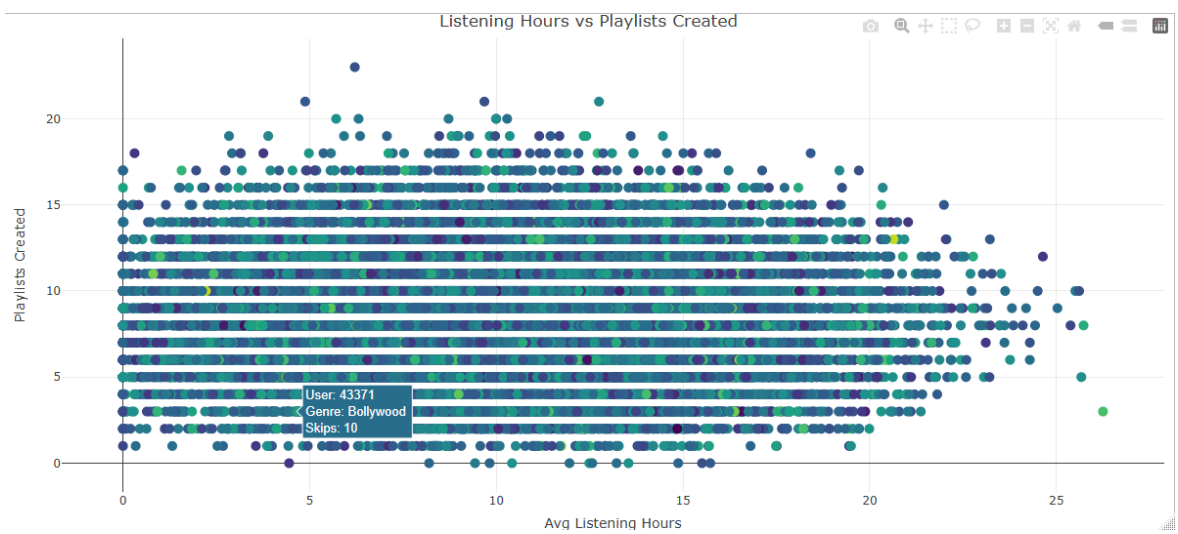
1. **Write a R program to demonstrate use of plotly library.**

```
library(ggplot2)
library(dplyr)

data <- read.csv("C:/Users/student/Desktop/TE/DAV/spotify_dataset.csv")

# 1. Interactive Bar: Avg Listening by Country
plot_ly(data, x = ~country, y = ~avg_listening_hours_per_week,
        type = 'bar', color = ~country) %>%
  layout(title = "Avg Listening Hours by Country",
         xaxis = list(title = "Country"),
         yaxis = list(title = "Avg Hours per Week"))
```



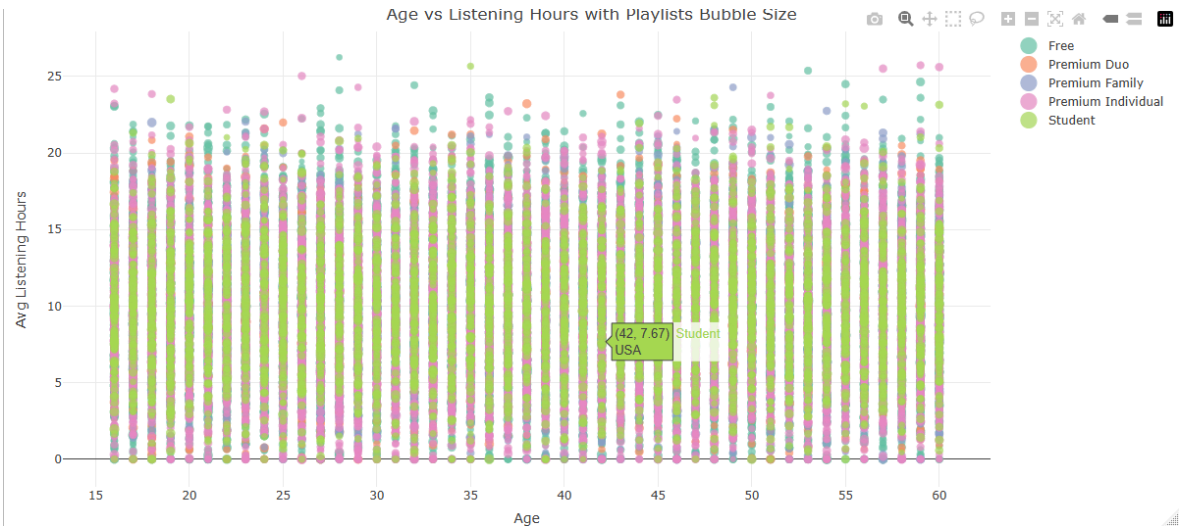
This plot compares how many hours people spend listening to music across different countries.

```
# 2. Interactive Scatter: Hours vs Playlists with hover info
plot_ly(data = data, x = ~avg_listening_hours_per_week, y =
~playlists_created,
        type = 'scatter', mode = 'markers',
        marker = list(size = 10, color= ~avg_skips_per_day, colorscale =
'Viridis'),
        hoverinfo = 'text',
        text = ~paste("User:", user_id,
                      "<br>Genre:", favorite_genre,
                      "<br>Skips:", avg_skips_per_day)) %>%
  layout(title = "Listening Hours vs Playlists Created",
         xaxis = list(title = "Avg Listening Hours"),
         yaxis = list(title = "Playlists Created"))
```



This plot looks for a relationship between how much someone listens (avg_listening_hours_per_week) and how many playlists they create.

```
# 3. Bubble chart: Age vs Avg Hours, bubble size = playlists created
  plot_ly(data, x = ~age, y = ~avg_listening_hours_per_week,
          size = ~playlists_created, sizes = c(10,50),
          color = ~subscription_type, text = ~country,
          type = 'scatter', mode = 'markers') %>%
   layout(title = "Age vs Listening Hours with Playlists Bubble Size",
           xaxis = list(title = "Age"),
           yaxis = list(title = "Avg Listening Hours"))
```



Conclusion:

The **ggplot2** implementation demonstrates the power of the Grammar of Graphics in creating high-quality, static visualizations. By using layers and faceting, it provides a structured, publication-ready way to compare Spotify user segments and identify statistical trends with high clarity.

The **plotly** implementation showcases the shift from static viewing to active data exploration. Through interactive tooltips, zooming, and multi-dimensional bubble charts, it allows for a deeper "drill-down" into individual user behaviors and complex correlations that static plots might miss.